

Part 14: Practice Problem Sets

Graded, with hints and no solutions

How to use this set

Rules that make the difference between practice and time-wasting:

1. **Answer the “think about” line before you write any code.** Out loud or on paper. That line is the actual exercise. The typing is not.
2. **Give yourself 20 minutes** on an Easy, 35 on a Medium. If you are stuck after that, read a hint, not a solution.
3. **After you solve it, say the pattern name out loud.** “That was bottom-up with a record and return split.” Naming is what makes the next problem faster.
4. **Redo, do not collect.** Solving 40 problems once teaches less than solving 20 problems twice, a week apart.
5. **Test on the ugly cases:** empty tree, single node, skewed tree, all negative values.

No solutions are printed here on purpose.

Level 1: Absolute Beginner

Get the template into your fingers. Every one of these is under ten lines.

- **Binary Tree Preorder Traversal** (Easy). <https://leetcode.com/problems/binary-tree-preorder-traversal/> Pattern: recursive DFS. Think: where does push_back go relative to the two calls, and why does the output vector need &?
- **Binary Tree Inorder Traversal** (Easy). <https://leetcode.com/problems/binary-tree-inorder-traversal/> Pattern: recursive DFS. Think: predict the output on our sample tree before coding.
- **Binary Tree Postorder Traversal** (Easy). <https://leetcode.com/problems/binary-tree-postorder-traversal/> Pattern: recursive DFS. Think: why is the root always last?
- **Maximum Depth of Binary Tree** (Easy). <https://leetcode.com/problems/maximum-depth-of-binary-tree/> Pattern: bottom-up. Think: what is the depth of an empty tree, and what do you add for yourself?
- **Sum of Left Leaves** (Easy). <https://leetcode.com/problems/sum-of-left-leaves/> Pattern: DFS with a flag. Think: a node cannot tell whether it is a left child. Who knows that, and how do you tell it?
- **Univalued Binary Tree** (Easy). <https://leetcode.com/problems/univalued-binary-tree/> Pattern: DFS returning a boolean. Think: what should NULL return, and why is it not false?

- **Search in a Binary Search Tree** (Easy). <https://leetcode.com/problems/search-in-a-binary-search-tree/> Pattern: BST descent. Think: why do you never need to look at both children?
- **Binary Tree Level Order Traversal** (Medium by label, Easy in practice). <https://leetcode.com/problems/binary-tree-level-order-traversal/> Pattern: BFS template. Think: why must the level size be snapshotted before the inner loop?

Level 2: Easy

Standard interview warm-ups. You should reach a point where these take under ten minutes each.

- **Invert Binary Tree** (Easy). <https://leetcode.com/problems/invert-binary-tree/> Pattern: simple DFS with modification. Think: if you swap first and then recurse, what must you be careful about?
- **Same Tree** (Easy). <https://leetcode.com/problems/same-tree/> Pattern: two-tree recursion. Think: what are the three base cases, and in which order must they be tested?
- **Symmetric Tree** (Easy). <https://leetcode.com/problems/symmetric-tree/> Pattern: two-tree recursion, mirrored. Think: why can you not just call Same Tree on the two halves?
- **Minimum Depth of Binary Tree** (Easy). <https://leetcode.com/problems/minimum-depth-of-binary-tree/> Pattern: DFS with a trap, or BFS with early exit. Think: what goes wrong on a tree that is one long right chain?
- **Path Sum** (Easy). <https://leetcode.com/problems/path-sum/> Pattern: top-down path. Think: where exactly is the target checked, at NULL or at a leaf? Why does it matter?
- **Balanced Binary Tree** (Easy). <https://leetcode.com/problems/balanced-binary-tree/> Pattern: bottom-up with a sentinel. Think: what is the cost of the naive version, and what single value can carry both height and failure?
- **Diameter of Binary Tree** (Easy). <https://leetcode.com/problems/diameter-of-binary-tree/> Pattern: bottom-up, record versus return. Think: which expression is recorded and which is returned?
- **Merge Two Binary Trees** (Easy). <https://leetcode.com/problems/merge-two-binary-trees/> Pattern: two-tree recursion. Think: what should happen when one of the two nodes is NULL?
- **Binary Tree Paths** (Easy). <https://leetcode.com/problems/binary-tree-paths/> Pattern: backtracking. Think: what must be undone when leaving a node?
- **Average of Levels in Binary Tree** (Easy). <https://leetcode.com/problems/average-of-levels-in-binary-tree/> Pattern: BFS template. Think: which type do you accumulate in, and why not int?

Level 3: Easy plus

Each of these needs two ideas at once, or an idea used slightly out of its comfort zone.

- **Binary Tree Level Order Traversal II** (Medium). <https://leetcode.com/problems/binary-tree-level-order-traversal-ii/> Pattern: BFS plus a final reversal. Think: is there any reason to be clever here?
- **Binary Tree Right Side View** (Medium). <https://leetcode.com/problems/binary-tree-right-side-view/> Pattern: BFS last of level, or DFS visiting right first. Think: why is “keep going right from the root” wrong?

- **Find Bottom Left Tree Value** (Medium). <https://leetcode.com/problems/find-bottom-left-tree-value/> Pattern: BFS. Think: is there a queue order that makes this a one-liner?
- **Sum Root to Leaf Numbers** (Medium). <https://leetcode.com/problems/sum-root-to-leaf-numbers/> Pattern: top-down accumulation. Think: what number do you pass down, and what does NULL contribute?
- **Path Sum II** (Medium). <https://leetcode.com/problems/path-sum-ii/> Pattern: backtracking plus target. Think: two things change as you descend. What are they?
- **Count Good Nodes in Binary Tree** (Medium). <https://leetcode.com/problems/count-good-nodes-in-binary-tree/> Pattern: top-down with carried maximum. Think: what does each node need to be told by its parent?
- **Subtree of Another Tree** (Easy). <https://leetcode.com/problems/subtree-of-another-tree/> Pattern: two-tree recursion inside a traversal. Think: why is this one of the rare cases where calling a helper at every node is acceptable?
- **Cousins in Binary Tree** (Easy). <https://leetcode.com/problems/cousins-in-binary-tree/> Pattern: BFS or DFS with depth and parent. Think: two facts must match and one must differ. Which?
- **Maximum Level Sum of a Binary Tree** (Medium). <https://leetcode.com/problems/maximum-level-sum-of-a-binary-tree/> Pattern: BFS with an accumulator. Think: what tie-breaking rule does the problem impose?

Level 4: Medium

These are the recurring interview patterns. Do not skip the “think about” lines here.

- **Binary Tree Zigzag Level Order Traversal** (Medium). <https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/> Pattern: BFS plus alternate reversal. Think: what happens if you try to alternate by pushing children in a different order instead?
- **Lowest Common Ancestor of a Binary Tree** (Medium). <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/> Pattern: bottom-up search. Think: state in one sentence what the function returns for an arbitrary node. Do not write code before you can.
- **Lowest Common Ancestor of a BST** (Medium). <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/> Pattern: BST descent. Think: what property identifies the split point?
- **Construct Binary Tree from Preorder and Inorder Traversal** (Medium). <https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/> Pattern: divide and conquer with index bookkeeping. Think: which array gives the root, which gives the split, and why must you build left before right?
- **Construct Binary Tree from Inorder and Postorder Traversal** (Medium). <https://leetcode.com/problems/construct-binary-tree-from-inorder-and-postorder-traversal/> Pattern: same, mirrored. Think: which end of postorder do you consume, and which subtree comes first now?
- **Validate Binary Search Tree** (Medium). <https://leetcode.com/problems/validate-binary-search-tree/> Pattern: range passed downward, or inorder check. Think: build a tree where the local check succeeds and the tree is not a BST.
- **Kth Smallest Element in a BST** (Medium). <https://leetcode.com/problems/kth-smallest-element-in-a-bst/> Pattern: inorder with a counter. Think: can you stop early? What does that do to the complexity?

- **Binary Tree Maximum Path Sum** (Hard, but a must-know). <https://leetcode.com/problems/binary-tree-maximum-path-sum/> Pattern: bottom-up, record versus return. Think: why $\max(0, \text{child})$, and why must the answer start at `INT_MIN`?
- **Path Sum III** (Medium). <https://leetcode.com/problems/path-sum-iii/> Pattern: prefix sums on a path. Think: what does the map hold, and when is an entry removed?
- **All Nodes Distance K in Binary Tree** (Medium). <https://leetcode.com/problems/all-nodes-distance-k-in-binary-tree/> Pattern: parent map plus BFS. Think: why can plain DFS from the target not work directly?
- **Vertical Order Traversal of a Binary Tree** (Hard label, Medium idea). <https://leetcode.com/problems/vertical-order-traversal-of-a-binary-tree/> Pattern: horizontal distance plus sorting. Think: what are the three tie-breaking keys, in order?
- **Serialize and Deserialize Binary Tree** (Hard label, Medium idea). <https://leetcode.com/problems/serialize-and-deserialize-binary-tree/> Pattern: preorder with null markers. Think: why does a plain traversal without markers lose the tree?
- **Flatten Binary Tree to Linked List** (Medium). <https://leetcode.com/problems/flatten-binary-tree-to-linked-list/> Pattern: preorder rearrangement. Think: which traversal order does the flattened list follow, and where do you attach the old right subtree?
- **Maximum Width of Binary Tree** (Medium). <https://leetcode.com/problems/maximum-width-of-binary-tree/> Pattern: BFS carrying an index. Think: indices grow like 2^i and 2^{i+1} . What goes wrong for a deep tree, and how do you fix it?
- **Longest Univalue Path** (Medium). <https://leetcode.com/problems/longest-univalue-path/> Pattern: diameter skeleton with a condition. Think: when does a child contribute nothing?

The three view problems

These are standard in Indian campus interviews and on GeeksforGeeks rather than LeetCode. They are worth doing.

- **Top View of Binary Tree.** <https://www.geeksforgeeks.org/problems/top-view-of-binary-tree/1> Pattern: BFS with horizontal distance, first node per column.
- **Bottom View of Binary Tree.** <https://www.geeksforgeeks.org/problems/bottom-view-of-binary-tree/1> Pattern: same, last node per column.
- **Boundary Traversal of Binary Tree.** <https://www.geeksforgeeks.org/problems/boundary-traversal-of-binary-tree/1> Pattern: three separate walks. Think: where can duplicates appear?

Must Solve

If you only have time for one list, this is it. Twenty problems covering every pattern in the course. Solve all of them, then redo the starred ones a week later.

#	Problem	Pattern tested
1	Binary Tree Preorder Traversal (144)	preorder, recursion
2	Binary Tree Inorder Traversal (94) *	inorder, iterative too
3	Binary Tree Postorder Traversal (145)	postorder, iterative too

4	Binary Tree Level Order Traversal (102) *	BFS template
5	Maximum Depth of Binary Tree (104)	bottom-up basics
6	Minimum Depth of Binary Tree (111)	the leaf trap
7	Invert Binary Tree (226)	simple DFS with change
8	Same Tree (100)	two-tree recursion
9	Symmetric Tree (101) *	mirror recursion
10	Balanced Binary Tree (110) *	sentinel, $O(N)$ rewrite
11	Diameter of Binary Tree (543) *	record versus return
12	Path Sum (112)	top-down path
13	Path Sum II (113)	backtracking
14	Binary Tree Paths (257)	backtracking, undo
15	Binary Tree Right Side View (199)	level based view
16	Binary Tree Zigzag Level Order (103)	BFS variation
17	Lowest Common Ancestor (236) *	ancestor recursion
18	Construct from Preorder and Inorder (105) *	construction
19	Binary Tree Maximum Path Sum (124) *	bottom-up, hard version
20	Vertical Order Traversal (987)	horizontal distance
21	Top View of Binary Tree (GFG)	horizontal distance
22	Bottom View of Binary Tree (GFG)	horizontal distance
23	Boundary Traversal (GFG)	three-part walk
24	Validate Binary Search Tree (98)	range passed downward
25	Serialize and Deserialize (297)	preorder with markers

Coverage check: DFS, BFS, all three traversal orders, iterative traversal (problems 2 and 3, do both ways), height, diameter, balance, paths, symmetry, inversion, LCA, views, vertical order, construction, BST, and bottom-up thinking. If you can solve all twenty-five without looking anything up, you know binary trees.

A four week plan

Week	Sections to study	Problems
1	Parts 1, 2, 3	Level 1, plus Must Solve 1 to 6
2	Parts 4, 5, 6	Level 2, plus Must Solve 7 to 11
3	Parts 7, 8, 9, 10	Level 3, plus Must Solve 12 to 19
4	Parts 11, 12, 13	Level 4, plus Must Solve 20 to 25

Then redo the starred problems from memory. If any of them takes more than 15 minutes, reread the relevant part.